

Smart School Bus Route Optimization System

A graph-based system that plans multi-stop school bus routes under capacity, distance, and student-priority constraints.

01

Inputs

Stops, coordinates, road distances, capacity, priorities

02

Graph

Validated adjacency-list representation

03

DAA engine

Dijkstra, Greedy, and BFS work together

04

Route output

Assignments, metrics, and live audit trail



System Architecture

The dashboard feeds a service layer, which runs graph algorithms and returns auditable route decisions.

01 User dashboard

Configure stops, bus capacity, student priorities.

02 React interface

Collect inputs; present route results.

03 Flask service

Validate requests; coordinate execution.

04 Graph processing + DAA algorithms

Builds the graph, allocates capacity, checks paths, computes routes.

05 Database

Stores inputs, assignments, metrics, node logs.

06 Live audit output

Shows route state, traversal order, objective cost, constraint status.



Input and Graph Representation

Operational inputs become a validated graph that the algorithms can traverse and optimize.

Inputs

- Stop coordinates
- Road distances
- Bus capacity
- Student priorities



Graph build

- Stops become nodes
- Road links become edges
- Distance becomes edge cost
- Inputs are validated



Adjacency list

Each node stores connected stops and their road distances.

Validated graph representation: the shared input layer for routing, allocation, and connectivity checks.



Three algorithms, three responsibilities

Each method answers a different routing question before the final result is returned.



Dijkstra

Finds lowest-cost paths using road distance as the edge cost.

PATH COST



Greedy

Allocates capacity while respecting student priorities.

ALLOCATION



BFS

Checks connectivity and minimum-hop reachability.

VALIDATION

They balance distance, capacity, priority, and connectivity constraints.



End-to-End System Workflow

The user moves from configuration to a route they can inspect and compare.



Configure

Enter stops, capacity, and student priorities.



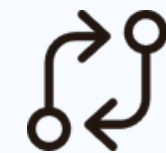
Execute

Build the graph and run the DAA core engine.



Audit

Review assignments, traversal order, and constraint status.



Compare

Evaluate route cost, hops, time, utilization, and satisfaction.



Route Output and Live Auditing

The result is more than a route: every decision can be traced from assignment to final metric.

Route output

Assignment view

- Bus assignment:** Students grouped by route and capacity.
- Traversal order:** Stops listed in visit sequence.
- Objective cost:** Distance-based route cost.
- Capacity utilization:** Used capacity shown for each bus.

Audit field	What it shows and why it matters	State
Node logs	Visited stops; explains traversal order	Live
Assignments	Students to buses; checks capacity and priority	Live
Metrics	Cost, hops, utilization; supports comparison	Final
Constraint status	Pass or review; confirms route validity	Final



User Journey and Performance Comparison

A clear journey connects the user’s inputs to a route they can evaluate against the same criteria.

User journey

- 1. Enter stops
- 2. Set bus capacity
- 3. Assign student priorities
- 4. Run optimization
- 5. Review and compare

Route option	Route cost	Hops	Execution time	Capacity utilization	Constraint satisfaction
Route output	Review	Review	Review	Review	Review
Alternative route	Compare	Compare	Compare	Compare	Compare
Selected route	Final	Final	Final	Final	Final

The framework keeps the comparison consistent without inventing benchmark values.

